Phase 5: Project Development Phase — Coding Implementation & Solution Algorithms

Project Name: Cab Booking (`UCab`)

Project ID: `N/A (Solo Track Submission)`

Team Member / Solo Developer: Shaik Sumiya Zainab

1. Core Algorithmic Implementations

1.1 Deterministic Distance & Total Fare Calculation (`BookCab.jsx`)

To compute trip pricing dynamically without relying on external mapping API quotas, the frontend calculates distance (`km`) and applies real-time deductions:

```
    // Dynamic Distance and Fare Calculation Algorithm inside BookCab.jsx:
    const calculateTripMetrics = (pickup, drop, car, promoCode, addRefreshments,
addCharity) => {
    // 1. Calculate deterministic simulated distance based on location string lengths & hash
    const hashDistance = Math.abs(pickup.length * 3 + drop.length * 2) % 35 || 12;
    const estimatedMinutes = Math.ceil(hashDistance * 2.5);
    // 2. Compute Base Cost
    let subtotal = car.baseFare + (hashDistance * car.pricePerKm);
    // 3. Apply Promo Code Deductions
    let discount = 0;
    if (promoCode === 'UCAB20') discount = subtotal * 0.20;
    else if (promoCode === 'WELCOME10') discount = subtotal * 0.10;
    // 4. Add Optional Customization Perks
    if (addRefreshments) subtotal += 50; // Chilled water & snacks
    if (addCharity) subtotal += 20;      // Driver welfare fund
    return {
    distanceKm: hashDistance,
    durationMin: estimatedMinutes,
    totalFare: Math.round(subtotal - discount),
    discountAmount: Math.round(discount)
    };
    };
```

1.2 Dual-Engine Transparent Fallback Algorithm (`carController.js`)

When handling REST API requests, controllers check the global database connection state and seamlessly execute Mongoose ORM commands or atomic file operations:

```
    // carController.js — Create Cab Implementation with Dual-Engine Failover:
    exports.createCar = async (req, res) => {
    try {
    const { name, cabType, pricePerKm, baseFare, seats, plateNumber, image } = req.body;
    if (global.isMongoConnected) {
    const newCar = await Car.create({ name, cabType, pricePerKm, baseFare, seats,
plateNumber, image });
    return res.status(201).json(newCar);
    } else {
    // Zero-Config JSON Fallback Execution
    const store = loadData();
    const newCar = {
    _id: 'car_' + Date.now(),
    name, cabType,
    pricePerKm: Number(pricePerKm),
    baseFare: Number(baseFare),
```

```
    seats: Number(seats),
    plateNumber,
    image: image || "https://images.unsplash.com/photo-1549317661-bd32c8ce0db2?
auto=format&fit=crop&w=600&q=80",
    isAvailable: true,
    createdAt: new Date().toISOString()
    };
    store.cars = store.cars || [];
    store.cars.unshift(newCar);
    saveData(store);
    return res.status(201).json(newCar);
    }
    } catch (error) {
    return res.status(500).json({ message: error.message });
    }
    };
```

1.3 JWT Role Validation & Zero-Config User Lookup (`authMiddleware.js`)

```
    // authMiddleware.js — Protect Route Interceptor:
    const protect = async (req, res, next) => {
    let token = req.headers.authorization?.split(' ')[1];
    if (!token) return res.status(401).json({ message: 'Not authorized, no token
provided' });
    try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET ||
'supersecretjwtkey_ucab_2026');
    if (global.isMongoConnected) {
    req.user = decoded.role === 'admin'
    ? await Admin.findById(decoded.id).select('-password')
    : await User.findById(decoded.id).select('-password');
    } else {
    const store = loadData();
    req.user = decoded.role === 'admin'
    ? (store.admins || []).find(a => a._id === decoded.id)
    : (store.users || []).find(u => u._id === decoded.id);
    }
    if (!req.user) return res.status(401).json({ message: 'Not authorized, user not
found' });
    req.userRole = decoded.role || 'user';
    next();
    } catch (error) {
    return res.status(401).json({ message: 'Not authorized, token verification failed' });
    }
    };
```